



Disciplina: Paradigmas de linguagem de programação

Professor: Munif Gebara Junior

Atividade 08

Maringá – PR
06/10/2026

1. Python

a)



```
main.py × + Console  
1 def adicionar(item, lista=[]):  
2     lista.append(item)  
3     return lista  
4  
5 print(adicionar(1))  
6 print(adicionar(2))  
7
```

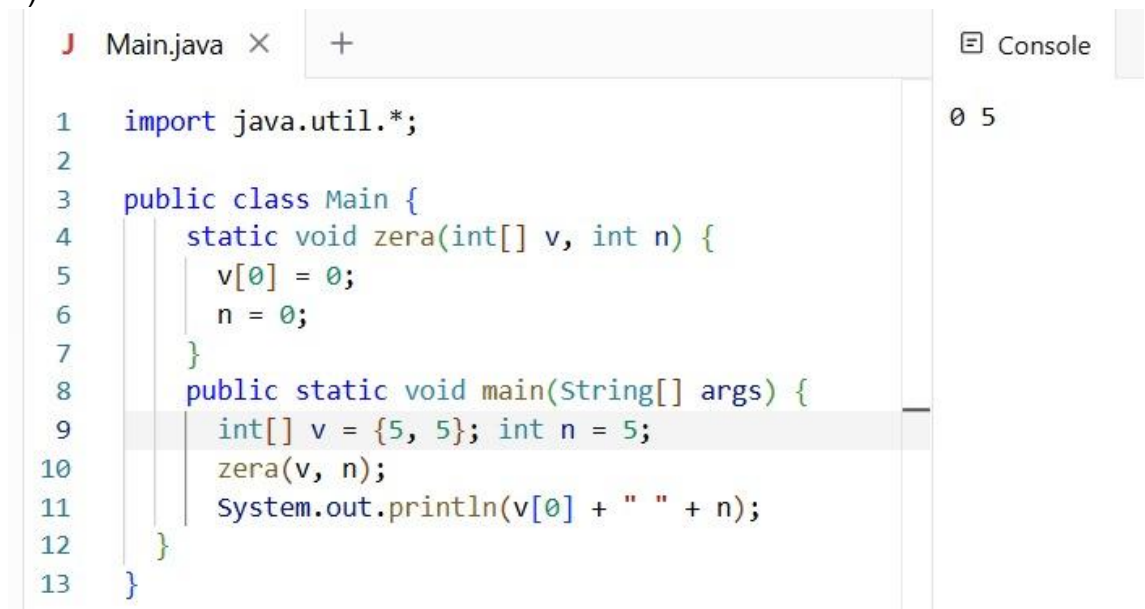
The console shows the output of the function calls: [1] and [1, 2].

b) Conceito: argumento padrão mutável.

A lista [] é criada apenas uma vez e é reutilizada nas chamadas seguintes. Por isso, o 2 é adicionado à mesma lista que já tinha o 1.

2. Java

a)



```
Main.java × + Console  
1 import java.util.*;  
2  
3 public class Main {  
4     static void zera(int[] v, int n) {  
5         v[0] = 0;  
6         n = 0;  
7     }  
8     public static void main(String[] args) {  
9         int[] v = {5, 5}; int n = 5;  
10        zera(v, n);  
11        System.out.println(v[0] + " " + n);  
12    }  
13 }
```

The console shows the output: 0 5.

b)

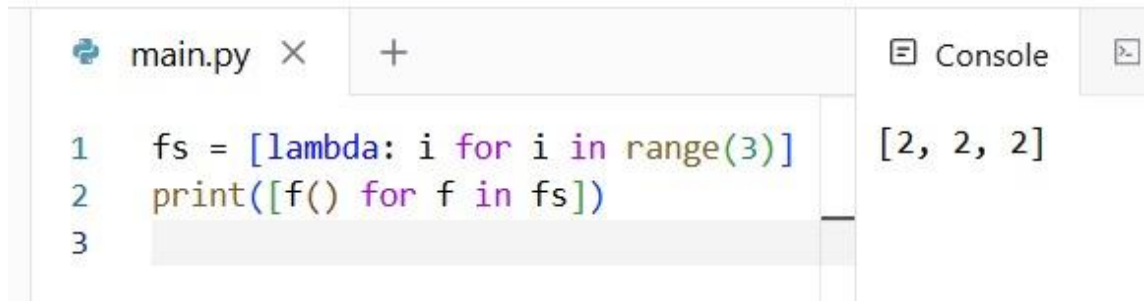
Conceito: passagem de parâmetros — referência de objetos/arrays e passagem por valor.

- v é um array. Ao passar v, o método consegue alterar o conteúdo original → v[0] vira 0.

- `n` é um `int` primitivo. O método recebe uma cópia de `n`, então `n = 0` não altera o `n` original.

3. Python

a)



```
main.py × + Console >_
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
3
```

[2, 2, 2]

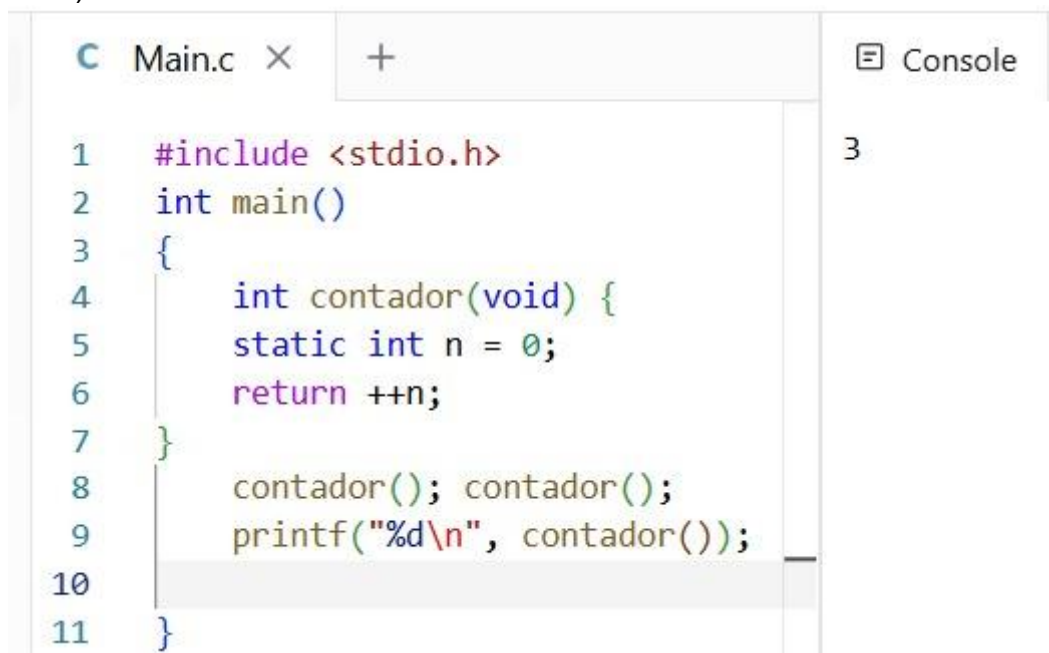
b)

Conceito: late binding (ligação tardia) em closures/lambda.

As lambda não guardam o valor de `i` no momento em que são criadas. Todas usam o valor de `i` quando são executadas. Como o `for` termina com `i = 2`, todas retornam 2.

4. C

a)



```
C Main.c × + Console
1 #include <stdio.h>
2 int main()
3 {
4     int contador(void) {
5         static int n = 0;
6         return ++n;
7     }
8     contador(); contador();
9     printf("%d\n", contador());
10
11 }
```

3

b)

Conceito: variável static local. A variável `n` mantém seu valor entre as chamadas da função.

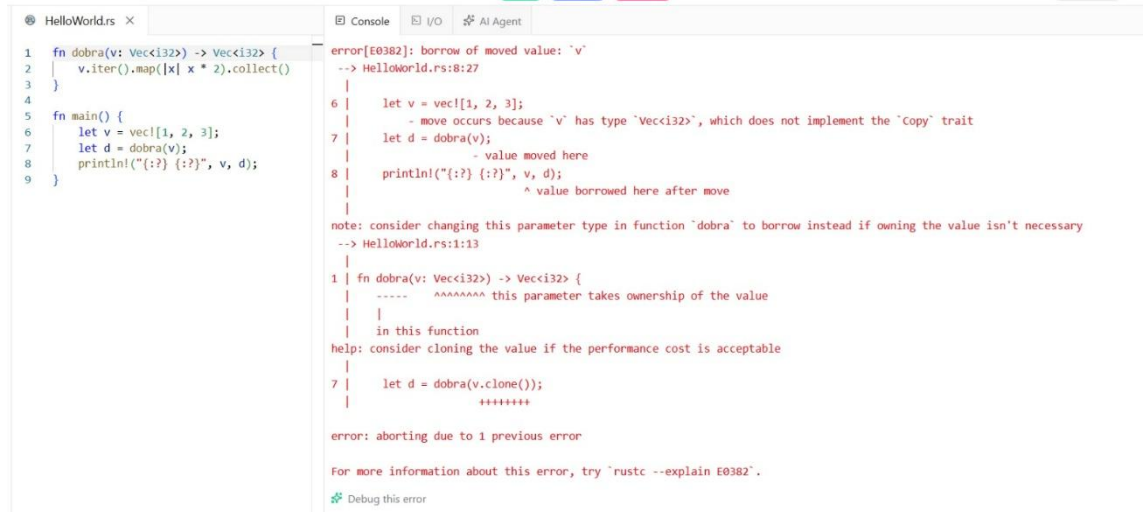
1ª chamada → `n = 1`

2ª chamada → `n = 2`

3ª chamada → n = 3
Por isso, o printf mostra 3.

5. Rust

a)



```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {  
2     v.iter().map(|x| x * 2).collect()  
3 }  
4  
5 fn main() {  
6     let v = vec![1, 2, 3];  
7     let d = dobra(v);  
8     println!("{:?} {:?}", v, d);  
9 }
```

error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:8:27
|
6 | let v = vec![1, 2, 3];
| - move occurs because `v` has type `Vec<i32>`, which does not implement the `Copy` trait
7 | let d = dobra(v);
| - value moved here
8 | println!("{:?} {:?}", v, d);
| ^ value borrowed here after move

note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary
--> HelloWorld.rs:1:13
1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
| ----- this parameter takes ownership of the value
| |
| in this function
help: consider cloning the value if the performance cost is acceptable
7 | let d = dobra(v.clone());
| ++++++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.
Debug this error

b)

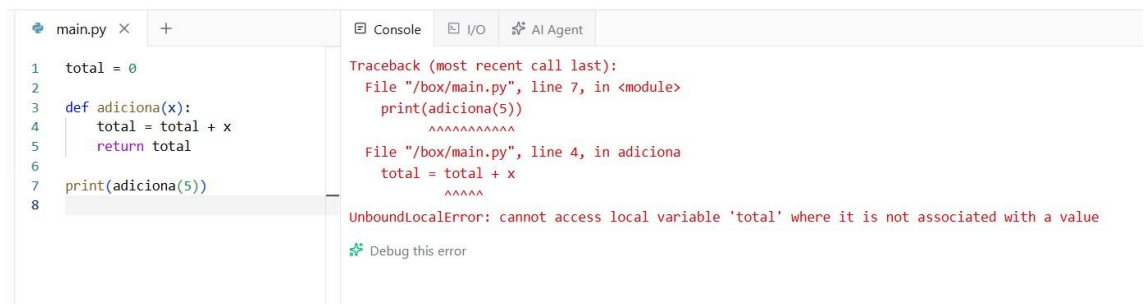
Conceito: ownership (propriedade).

Ao fazer: let d = dobra(v);

o v é movido para a função dobra. Depois disso, v não pode mais ser usado no println.

6. Python

a)



```
1 total = 0  
2  
3 def adiciona(x):  
4     total = total + x  
5     return total  
6  
7 print(adiciona(5))  
8
```

Traceback (most recent call last):
File "/box/main.py", line 7, in <module>
 print(adiciona(5))
 ^^^^^^^^^
File "/box/main.py", line 4, in adiciona
 total = total + x
 ^^^^^
UnboundLocalError: cannot access local variable 'total' where it is not associated with a value
Debug this error

b)

O código dá erro e não imprime um número.

Conceito: escopo de variáveis (local x global).

Dentro de adiciona(), quando aparece:

`total = total + x`

O Python entende total como uma variável local. Porém, ela ainda não recebeu valor dentro da função. O `total = 0` de fora é global e não é usado automaticamente.